

```
In [ ]: ## Support Intelligence & Cancellation Analysis
```

```
In [3]: # import Libraries
!pip install numpy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in C:\Users\KOUSHIK MONDAL\AppData\Roaming\Python\Python314\site-packages (2.5.3)

```
In [4]: print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
print("Seaborn:", sns.__version__)
print("SQLite: OK")
print("OpenPyXL: OK")
```

NumPy: 2.5.3
Pandas: 3.0.6
Matplotlib: 3.11.2
Seaborn: 0.13.2
SQLite: OK
OpenPyXL: OK

```
In [ ]: # Import Database
```

```
In [15]: excel_file = 'customer_churn_data_raw.xlsx'

# Create SQLite database connection
conn = sqlite3.connect('customer_churn.db')
excel_data = pd.ExcelFile(excel_file)

for sheet in excel_data.sheet_names:
    df = pd.read_excel(excel_file, sheet_name=sheet) # Read sheet into dataframe
    # Write dataframe to SQLite table
    df.to_sql(
        name=sheet,
        con=conn,
        if_exists='replace',
        index=False
    )
# Close connection
conn.close()
print("All sheets successfully converted into SQLite tables.")
```

All sheets successfully converted into SQLite tables.

```
In [16]: conn = sqlite3.connect('customer_churn.db')
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table'
        """
tables = pd.read_sql(sql_query, conn)
```

```
# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Create dataframe: df_{table_name}")

conn.close()
```

Create dataframe: df_db_customer

Create dataframe: df_db_subscription

Create dataframe: df_db_support

In []: *# Data Cleaning*

In [18]: df_db_customer.head()

Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody>
<tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [19]: df_db_customer.tail()

Out[19]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
</tbody> <tbody></tbody>

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [17]: # Print table names and column names
conn = sqlite3.connect('customer_churn.db')
for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

Table Name: db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

```
In [20]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [ ]: # a. rename col - name to customer_name
        # b. drop columns - interest and pincode
        # c. change data type - dob
        # d. data standardization - gender
        # e. fix missing values (using existing data) - country
```

```
In [25]: # a
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

```
In [26]: df_db_customer
```

```
Out[26]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>interests</th>
<th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th>
<th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
</tbody> <tbody> </tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None

0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [27]: # b
df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)
```

```
In [28]: df_db_customer
```

```
Out[28]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th>
<th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th>
<th>19</th> <th>20</th> </tbody> <tbody></tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

```
In [35]: df_db_customer.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   customerid      21 non-null    str
 1   customer_name   21 non-null    str
 2   country         18 non-null    str
 3   state           21 non-null    str
 4   gender          21 non-null    str
 5   dob             21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB

```

```

In [31]: # c
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])

```

```

In [37]: # d
# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo

```

```

In [40]: # e
# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]

```

```

Out[40]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>customer_name</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th>
<th>8</th> <th>12</th> </tbody> <tbody></tbody>

```

0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

```

In [43]: state_country_mapping = (
    df_db_customer
    .dropna(subset=['country'])
    .set_index('state')['country']
    .to_dict()
)

df_db_customer['country'] = df_db_customer['country'].fillna(
    df_db_customer['state'].map(state_country_mapping)
)

```

```

In [44]: df_db_customer['country'].isna().sum()

```

```

Out[44]: np.int64(0)

```

```

In [45]: df_db_customer['country']

```

```
Out[45]: 0      India
         1      India
         2      India
         3      India
         4      India
         5      India
         6      India
         7      India
         8      Nepal
         9      Nepal
        10      India
        11      India
        12      India
        13      India
        14      India
        15      India
        16      India
        17      India
        18      India
        19      India
        20      India
        Name: country, dtype: str
```

```
In [46]: df_db_subscription.head()
```

```
Out[46]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

```
In [47]: df_db_subscription.tail()
```

```
Out[47]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody>
<th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody>
<tbody></tbody>
```

0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	NaN	NaN	20.99	550	62
0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	NaN	NaN	13.99	840	27
0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	99
0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	NaN	NaN	22.99	1955	7
0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	NaN	NaN	13.99	790	47

```
In [52]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         21 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB
```

```
In [53]: df_db_subscription.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score             21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB

```

```

In [54]: date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    datetime64[us]
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    datetime64[us]
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     datetime64[us]
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score             21 non-null    int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB

```

```

In [56]: df_db_support.head()

```

```
Out[56]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

```
In [57]: # drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [ ]: # change data type to date - complaint_date
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```
In [ ]: # Feature Engineering
```

```
In [58]: # create a new col using existing column - churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_dat
df_db_subscription.head()
```

```
Out[58]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th>
</tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	C
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	C
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	C
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In []: `# JOINS`

In [61]: `# first fix support table duplicates then merge`
`df = (df_db_subscription`
 `.merge(df_db_customer, on = 'customerid', how= 'left')`
 `.merge(df_db_support, on = 'customerid', how= 'left') )`

In [62]: `df_db_subscription.shape`

Out[62]: (21, 12)

In [63]: `df.shape`

Out[63]: (23, 20)

In [64]: `df.head()`

Out[64]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe`
`tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>`
`<th></th> <th>customerid</th> <th>subscription_start_date</th>`
`<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>`
`<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>`
`<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th>`
`<th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th>`
`<th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th>`
`</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th>`
`</tbody> <tbody></tbody>`

0002-ORFBO	2021-03-15	Referral	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	C
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	C
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	C



In [65]: `df.tail()`

```
Out[65]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th>
<th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th>
<th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th>
</thead> <tbody> <th>18</th> <th>19</th> <th>20</th> <th>21</th>
<th>22</th> </tbody> <tbody> </tbody>
```

0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	NaT	NaN	13.99	840	27	0	
0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	99	1	r
0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	99	1	r
0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	NaT	NaN	22.99	1955	7	0	
0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	NaT	NaN	13.99	790	47	0	

```
In [71]: print('df_db_subscription unique value:', df_db_subscription['customerid'].nunique())
df_db_subscription unique value: 21
```

```
In [70]: print('df_db_customer unique value:', df_db_customer['customerid'].nunique())
df_db_customer unique value: 21
```

```
In [69]: print('df_db_support unique value:', df_db_support['customerid'].nunique())
df_db_support unique value: 7
```

```
In [67]: print('df_db_support all value:', df_db_support['customerid'].size)
df_db_support all value: 9
```

```
In [72]: df_db_support
```

```
Out[72]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody>
<tbody></tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60
------------	---------------------	---	----

0003-MKNFE	2024-08-28 00:00:00	Y	10
------------	---------------------	---	----

0013-EXCHZ	2024-01-20 00:00:00	Y	20
------------	---------------------	---	----

0013-MHZWF	2025-03-18 00:00:00	N	90
------------	---------------------	---	----

0013-SMEOE	2024-11-01 00:00:00	N	30
------------	---------------------	---	----

0017-IUDMW	2024-04-10 00:00:00	Y	25
------------	---------------------	---	----

0019-EFAEP	2024-09-27 00:00:00	Y	30
------------	---------------------	---	----

0022-TCJCI	2024-09-13 00:00:00	Y	10
------------	---------------------	---	----

0022-TCJCI	2024-09-14 00:00:00	N	90
------------	---------------------	---	----

```
In [73]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer
```

```
In [74]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
```

```
In [75]: df_db_support['customerid'].size
```

```
Out[75]: 7
```

```
In [76]: # merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left') )
```

```
In [77]: df.shape
```

```
Out[77]: (21, 21)
```

```
In [78]: # export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

```
In [79]: ## Data Analysis
```

```
In [80]: # Churn Rate
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [81]: # Retenion Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [82]: # Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [87]: # Churned users by State + Revenue + User Count
churn_by_state = (df[df['churn_flag'] == 1].groupby('state').agg(revenue=('month
print(churn_by_state)
```

	state	revenue	user_count
0	Delhi	13.99	1
1	Karnataka	20.98	2
2	Meghalaya	21.98	2
3	Telangana	16.99	1

```
In [89]: # Churn by subscription type + sum(revenue) & count of users
churn_by_subscription = (df[df['churn_flag'] == 1].groupby('subscription_type').
print(churn_by_subscription)
```

	subscription_type	revenue	user_count
0	Paid	12.99	1
1	Refferal	60.95	5

```
In [90]: # ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

ARPU = 18.85

```
In [94]: df.columns
```

```
Out[94]: Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
'tenure_days'],
dtype='str')
```

```
In [97]: df.head(2)
```

```
Out[97]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>customerid</th> <th>subscription_start_date</th>
<th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th>
<th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th>
<th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>country</th>
<th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th>
<th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>age</th>
<th>tenure_days</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	2021-03-15	Referral	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	Ind
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Ind

<p>2 rows × 23 columns</p>



```
In [92]: # Calculate customer age
df['age'] = (
    pd.Timestamp.today().year - pd.to_datetime(df['dob']).dt.year
)
print(df[['customerid', 'dob', 'age']].head())
```

	customerid	dob	age
0	0002-ORFBO	1982-04-12	44
1	0003-MKNFE	1995-11-23	31
2	0004-TLHLJ	1978-02-15	48
3	0011-IGKFF	2001-08-30	25
4	0013-EXCHZ	1990-05-05	36

```
In [100... pd.Timestamp.today()
```

```
Out[100... Timestamp('2026-09-30 16:16:05.412466')
```

```
In [93]: # Avg Customer Tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)
avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

Avg Tenure (Days) = 1550 0

```
In [95]: # Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

Revenue at Risk (Rs 'K') = 73.94

```
In [96]: # Escalation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

```
In [101... # Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

Avg Compliants Per User = 0.43

```
In [102... # Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int
corr_df = df[['escalations', 'churn_flag']].dropna()
correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation,2))
```

Correlation between escalation vs churn is = 0.77

```
In [104... # Create a column using existing col - Churn risk
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50 & (df['churn_score'] < 70)),
    (df['churn_score'] >= 70)
]
choices = ['low', 'med', 'high']
df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

```
In [105... df.columns
```

```
Out[105... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
      'tenure_days', 'churn_risk'],
      dtype='str')
```

```
In [109... df[['churn_risk', 'churn_score']].head()
```

```
Out[109... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>churn_risk</th> <th>churn_score</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

```
low 12
```

```
high 91
```

```
low 34
```

```
low 8
```

```
high 88
```

```
In [ ]: ##Visualization using Matplotlib
```

```
In [120... df_visual = df.copy()
```

```
In [125... df_visual.columns
```

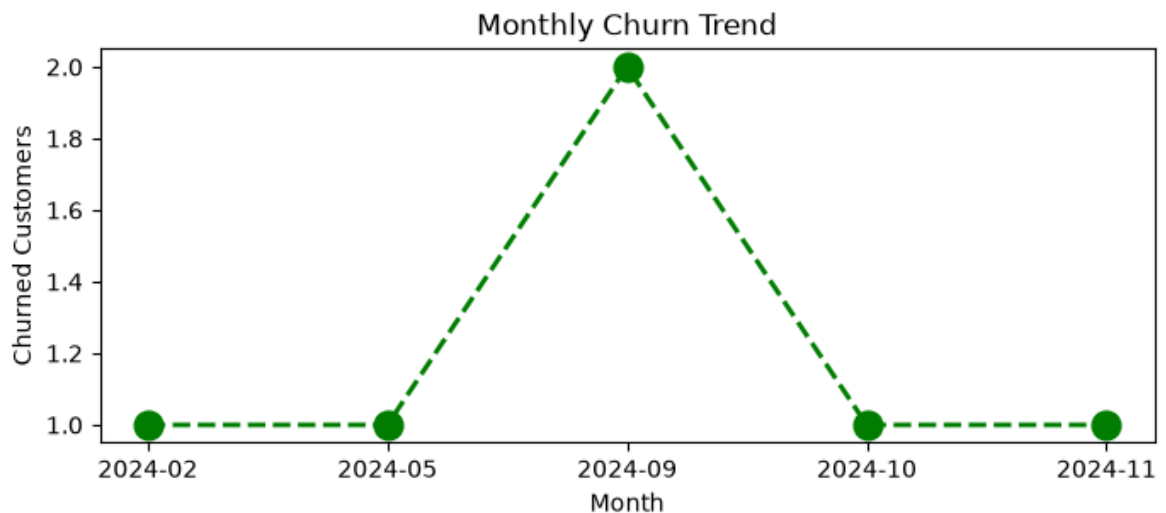
```
Out[125... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
      'tenure_days', 'churn_risk'],
      dtype='str')
```

```
In [126... # Monthly Churn Trend (Time Series KPI)
```

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o')

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

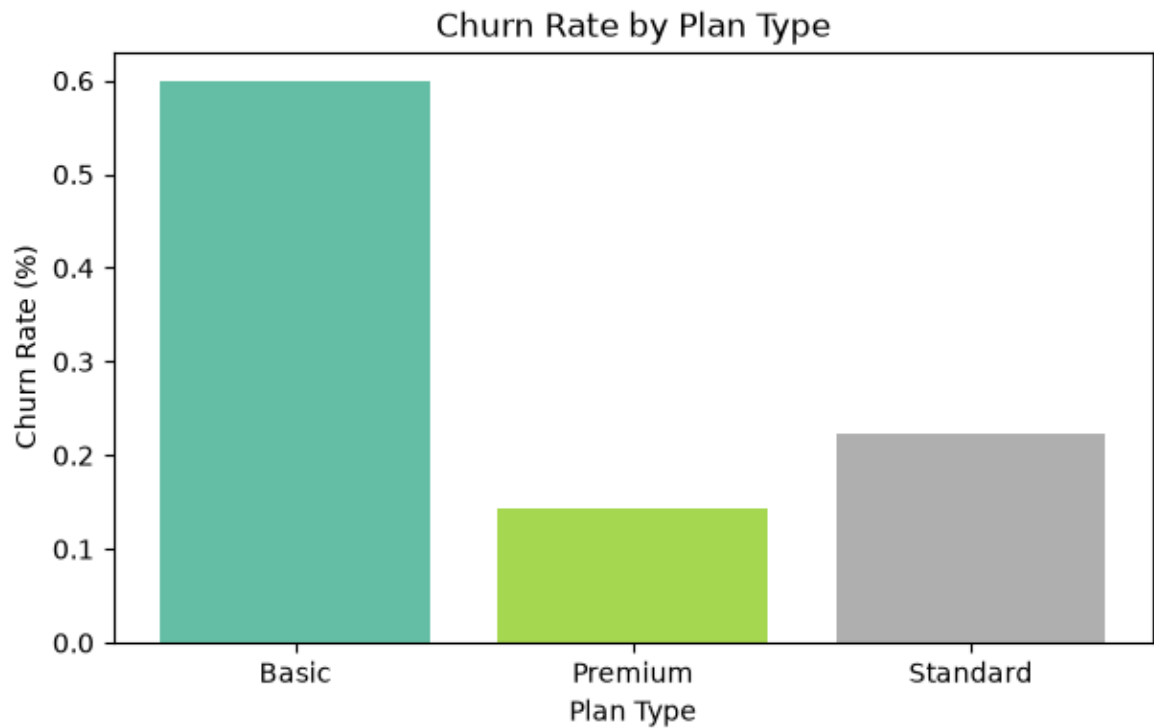


```
In [130... # Churn Rate by Plan type
```

```
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()
# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

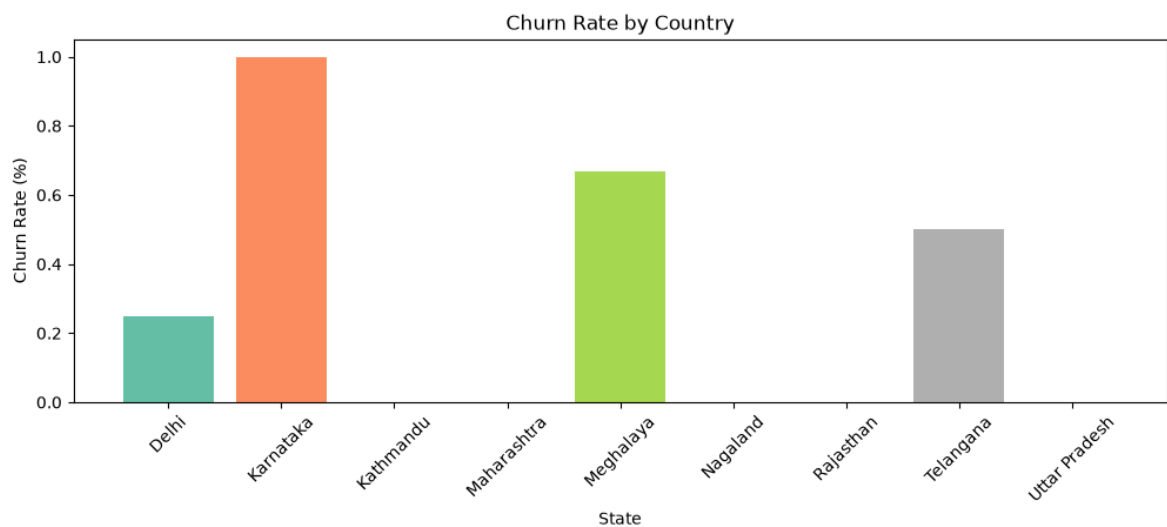
plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```



```
In [131... # Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()
# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



```
In [132... ## Visulaization using Seaborn
```

```
In [133... # encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

```
Out[133... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
      'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

```
In [134... df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_ris
```

```
Out[134... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

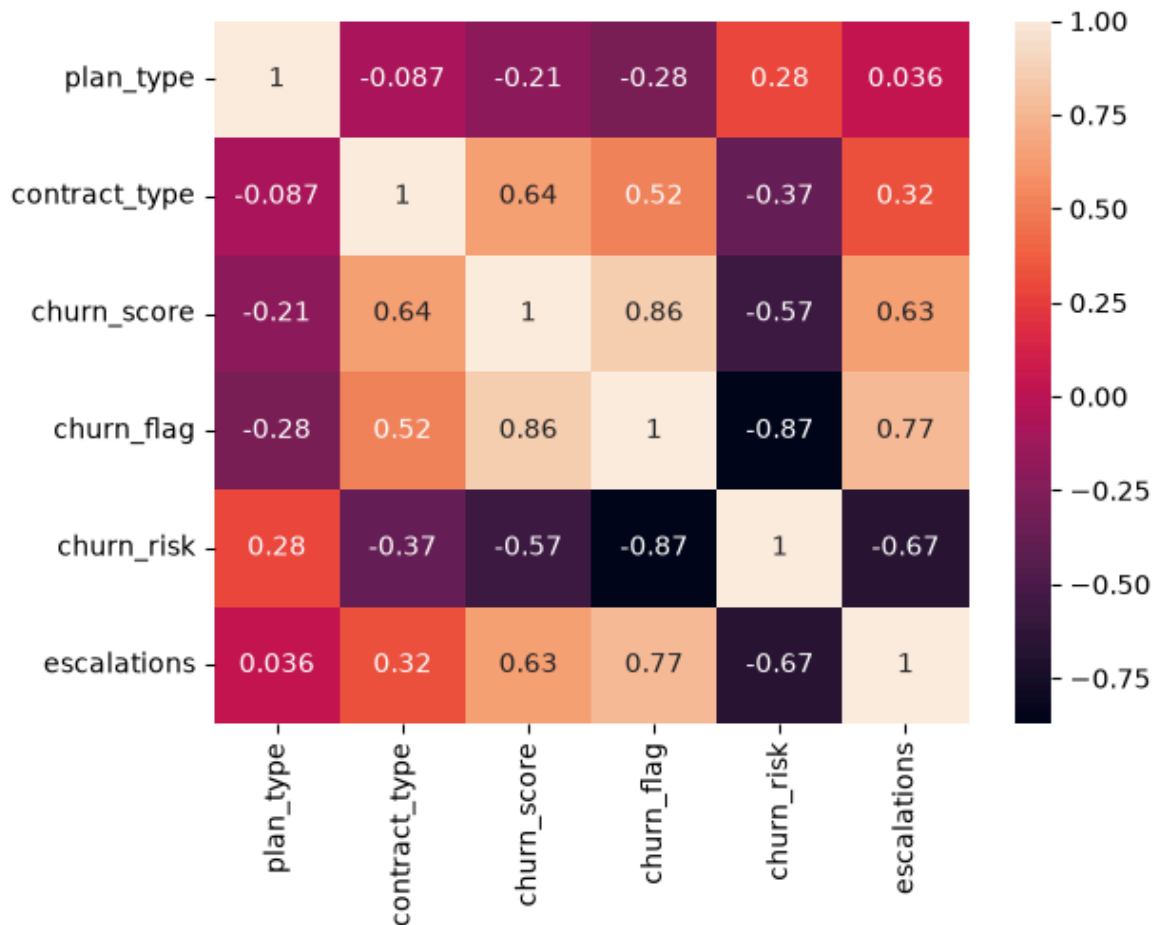
Standard	Annual	12	0	low	0
Premium	Annual	91	1	high	1
Basic	Monthly	34	0	low	0
Premium	Annual	8	0	low	0
Standard	Monthly	88	1	high	1

```
In [135... # remove warnings
import warnings
warnings.filterwarnings("ignore")
```

```
In [136... # incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']
for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
In [137... # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[137... <Axes: >
```



```
In [138... print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : <StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
contract_type : <StringArray>
['Annual', 'Monthly']
Length: 2, dtype: str
churn_risk : <StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```

```
In [139... # Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]
order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk' : ['low', 'med', 'high']
}
for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order, ordered=True)
```

```
In [140... df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]
```

Out[140... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

Standard	Annual	12	0	low	0
----------	--------	----	---	-----	---

Premium	Annual	91	1	high	1
---------	--------	----	---	------	---

Basic	Monthly	34	0	low	0
-------	---------	----	---	-----	---

Premium	Annual	8	0	low	0
---------	--------	---	---	-----	---

Standard	Monthly	88	1	high	1
----------	---------	----	---	------	---

In [141... df_encoded.head()

Out[141... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1	1	12	0	0	0
---	---	----	---	---	---

2	1	91	1	2	1
---	---	----	---	---	---

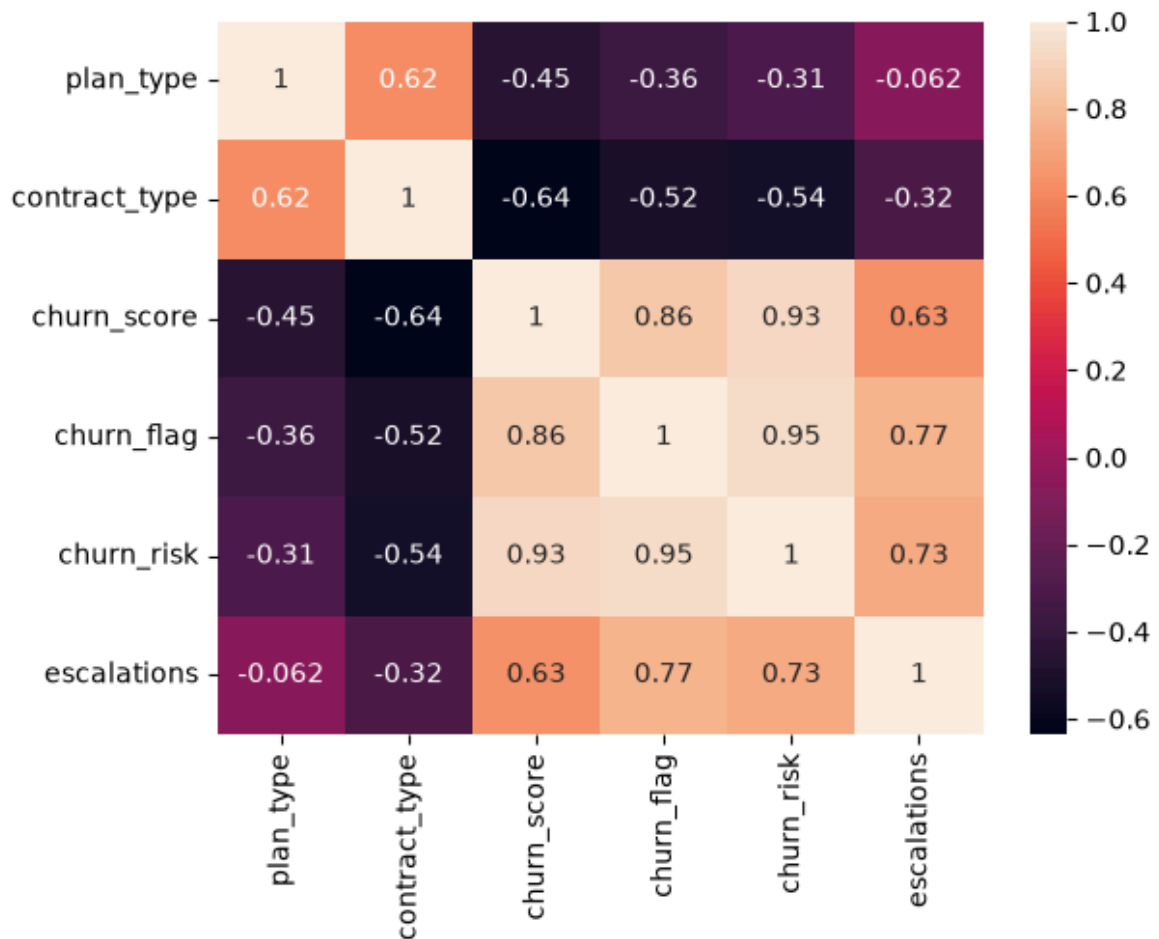
0	0	34	0	0	0
---	---	----	---	---	---

2	1	8	0	0	0
---	---	---	---	---	---

1	0	88	1	2	1
---	---	----	---	---	---

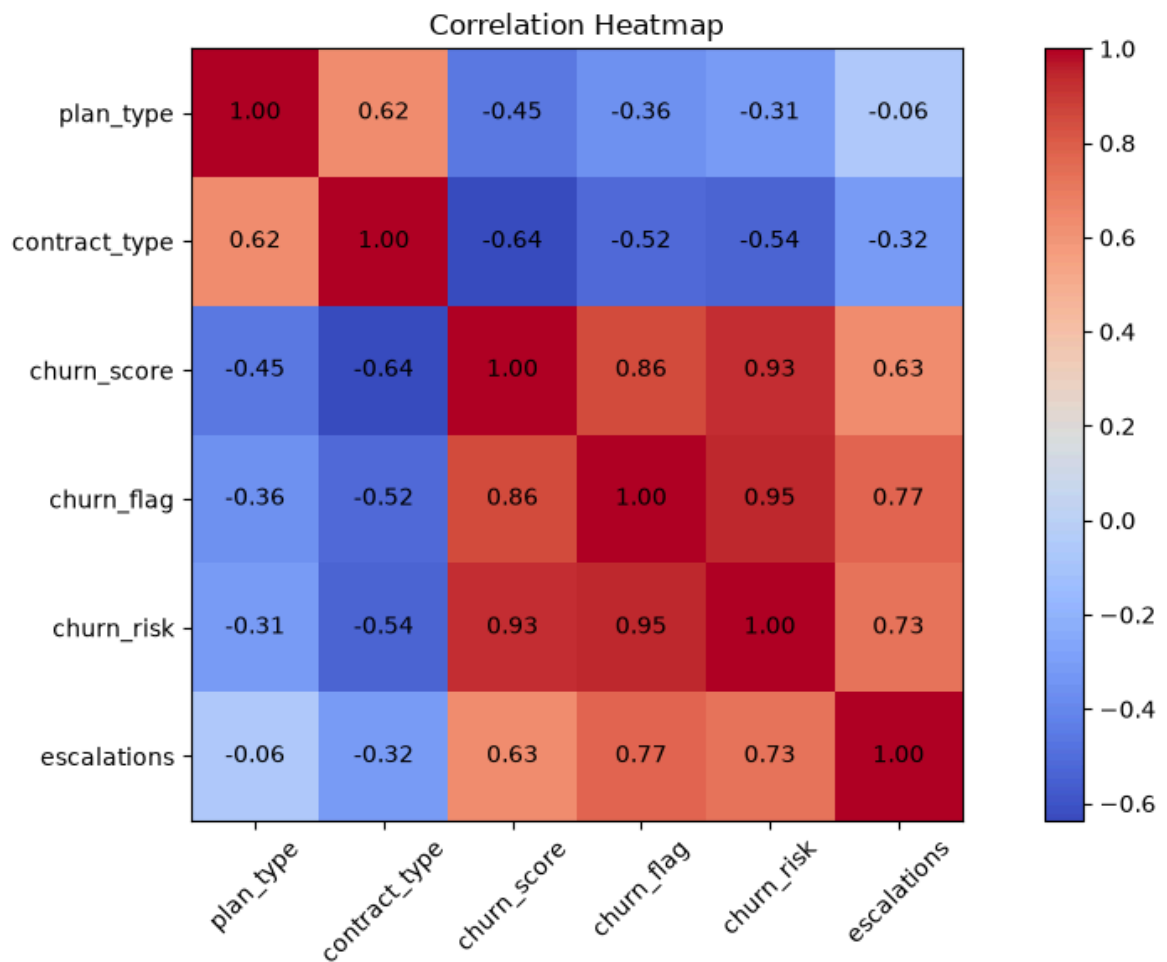
In [142... # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)

Out[142... <Axes: >



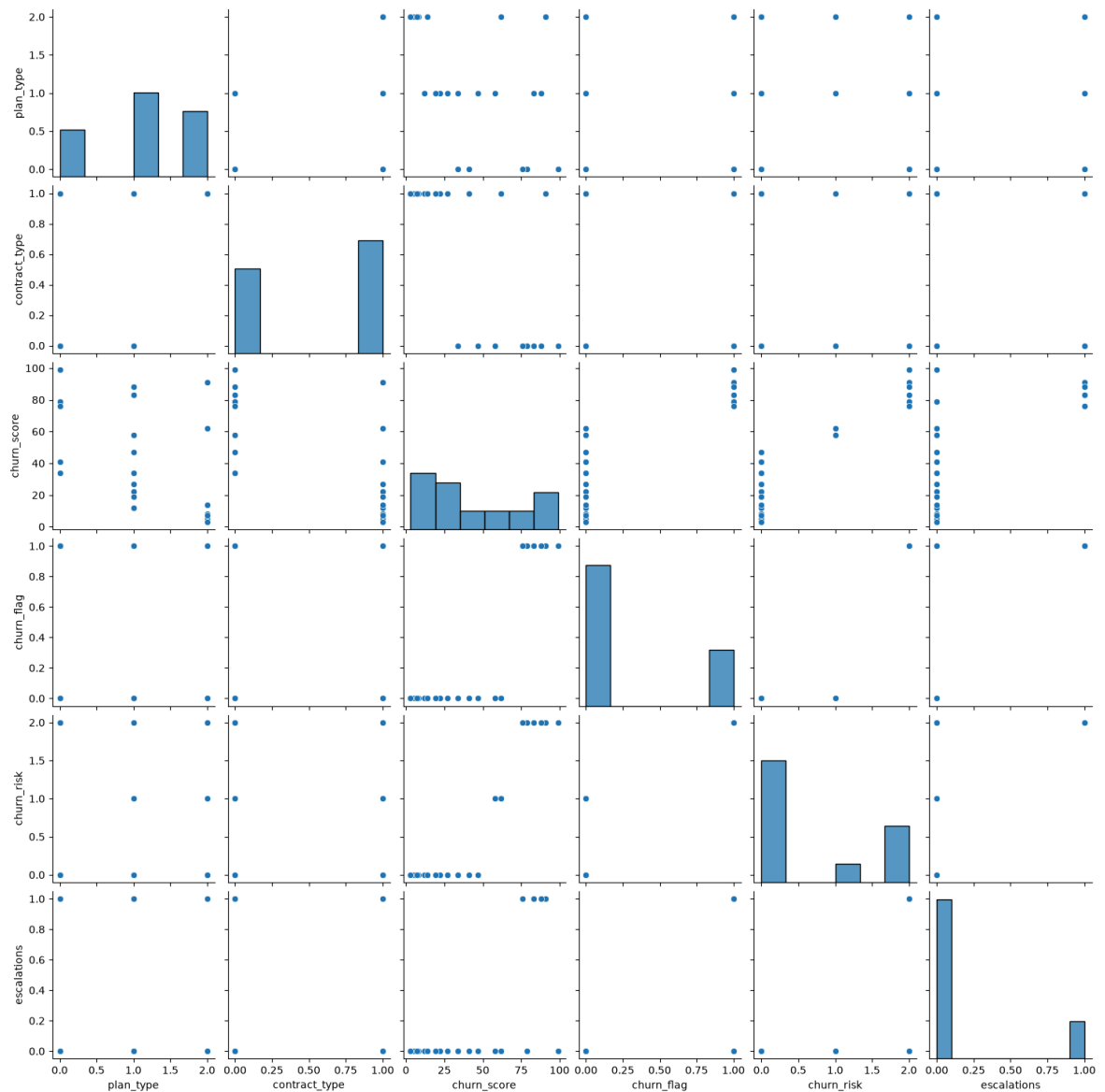
```
In [143... # Heatmap Using Matplotlib - difficult to plot it
corr_matrix = df_encoded.corr()
fig, ax = plt.subplots(figsize=(10, 6))
cax = ax.imshow(corr_matrix, cmap='coolwarm')
fig.colorbar(cax)
# Axis Labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)
# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )
plt.title('Correlation Heatmap') # Title
plt.tight_layout()
plt.show()
```



```
In [144... # pairplot - Plot pairwise relationships in a dataset
sns.pairplot(df_encoded)
```

```
Out[144... <seaborn.axisgrid.PairGrid at 0x23854676510>
```

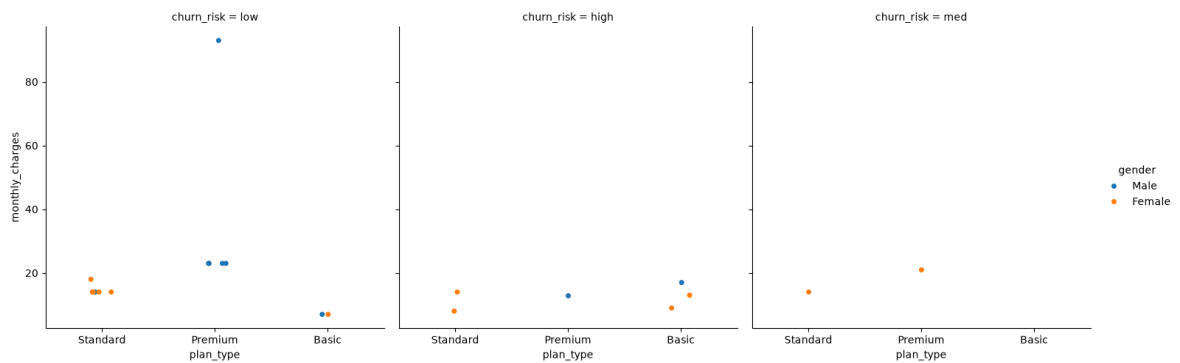


In [145... `df_visual.columns`

Out[145... `Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age', 'tenure_days', 'churn_risk', 'cancellation_month'], dtype='str')`

In [146... `# catplt/Facegrid plot - multi-dim comparison`
`sns.catplot(data=df_visual,`
`x='plan_type',`
`y='monthly_charges',`
`hue='gender',`
`col='churn_risk')`

Out[146... `<seaborn.axisgrid.FacetGrid at 0x23855e03230>`



In [147... *## Pivot table*

```
In [148... pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
)
```

Out[148... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>churn_flag</th> <th>plan_type</th> <th></th> </thead> <tbody>
<th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000
0.142857
0.222222

```
In [149... # pivot table using multiple cols and agg type
pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
)
```

Out[149... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th>
<th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody>
<th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000 5 52.95
0.142857 7 218.93
0.222222 9 123.91

In [150... *# Project Completed*

In []: